

Encoding Reversible Petri nets into CCSK^{*}

Hernán C. Melgratti¹, Claudio Antares Mezzina², and G. Michele Pinna³

¹ ICC - Universidad de Buenos Aires, Argentina

² Dipartimento di Scienze Pure e Applicate, Università di Urbino, Urbino, Italy

³ Dipartimento di Matematica e Informatica, Università di Cagliari, Italy

Abstract. Reversibility in computational models is a crucial aspect for applications such as fault-tolerant computing, distributed systems, and quantum computing. Petri nets provide a well-established formalism for modeling concurrent systems, while CCS (Calculus of Communicating Systems) and its reversible extension, CCSK, offer a process algebraic approach to system specification. In this work, we define a method to encode reversible Petri nets into a CCSK, ensuring that the structural properties and execution semantics of the original Petri nets are preserved in the translation. We build upon previous research on encoding standard Petri nets into CCS and extend it by incorporating communication keys to track causal dependencies, enabling reversibility.

1 Introduction

In a reversible system, computation can proceed in two directions: forward, executing standard operations, and backward, reversing the effects of the forward computation. Originally, reversible computing was explored for its potential to enable low-energy computation [3]. While energy efficiency remains a relevant concern, reversibility has gained significant attention due to its applications in various domains, such as fault-tolerant computing [10,7,27,39], distributed systems, biochemical systems [12,22,31], and quantum computing. In these contexts, the ability to reverse computations enables error recovery, state exploration, and process rollback, making reversible models crucial for designing robust and resilient systems. Among the formalisms used to describe concurrent systems, Petri nets [33] offer a well-established graphical representation, while process calculi, such as CCS [28] (Calculus of Communicating Systems), provide an algebraic approach to describing interactions and process behaviors.

In the last decades, many efforts have been made to endow computation models with reversible semantics [2,26]. In particular, two different models have been proposed for CCS: reversible CCS (RCCS) [6,11] and CCS with communication keys (CCSK) [32]. Both of them incorporate a logging mechanism in the operational semantics of CCS that enables the undoing of computation steps.

^{*} This work has been supported by the Italian MUR PRIN 2022 project *DeKLA*, the INDAM-GNCS project CUP_E53C24001950001 *MARQ*, and the European Union - NextGenerationEU program Research and Innovation Program PE00000014 *Security and Rights in the CyberSpace* (SERICS), projects STRIDE and SWOPS

Moreover, it has been shown that they are isomorphic [15] since they only differ on how they log information about past computations: while RCCS relies on some form of memory/monitor, CCSK uses keys. On the Petri nets side there exist several attempts to give a reversible semantics to nets [25,24,23,30,29], with just [24] providing a suitable class of Petri nets for RCCS.

CCS nets [9] are a subclass of Petri nets aiming at capturing the behavior of CCS. Recently an encoding of CCS nets into CCS processes has been provided [4] showing that a CCS net is strongly bisimilar to the generated CCS process. This paper is motivated by the following question: what is a suitable subclass of Petri nets for CCSK? To answer to this question we start from CCS nets and show what are the ingredients to first obtain key labelled nets, that is CCS nets enriched with (i) extra places indicating that a transition has fired and with (ii) particular token recording the causal path of a computation. Then we show how it is possible to add reversibility to such kind of nets obtaining CCSK nets. Finally we show how it is possible to generate a CCSK term starting from a CCSK net. This will enable us to obtain a suitable sub-class of (reversible) Petri nets corresponding to CCSK processes.

A Tribute to Jean-Bernard Stefani. This paper is an occasion to celebrate Jean-Bernard Stefani 65th birthday. Jean-Bernard Stefani has made significant contributions to theoretical computer science, particularly in concurrency theory [21,20,19], process calculi [37,34,35], and component-based software engineering [5,1,36,38]. His research has also advanced the field of reversible computation in concurrent systems with the idea of modelling and verifying fault-handling systems [17,16,14,18,39], as well as enabling reverse debugging techniques [13,8]. Through his insights, passion, and enthusiasm for computer science—and science more broadly—Stefani has inspired new research directions and fostered generations of computer scientists.

2 CCS and CCSK

Let $\mathcal{A} = \{a, b, c, \dots\}$ be a set of *actions*, and let $\overline{\mathcal{A}} = \{\bar{a} \mid a \in \mathcal{A}\}$ be the set of their corresponding *co-actions*. The set of all possible actions is given by $\mathbf{Act} = \mathcal{A} \cup \overline{\mathcal{A}}$. This set is extended to $\mathbf{Act}_\tau = \mathbf{Act} \cup \{\tau\}$, where $\tau \notin \mathbf{Act}$ is a distinguished *silent* (or internal) action. We use α, β to range over elements of $\mathbf{Act}_\tau$. We assume that for each $\alpha \in \mathbf{Act}$, $\overline{\overline{\alpha}} = \alpha$ holds. The syntax of CCS is presented below.

$$\begin{aligned} \text{(Actions)} \quad \alpha &::= a \mid \bar{a} \mid \tau \\ \text{(Processes)} \quad P &::= \mathbf{0} \mid \alpha.Q \mid P + P' \\ \text{(Agents)} \quad Q &::= P \mid Q \parallel Q' \mid (\nu a)Q \mid X \end{aligned}$$

In CCS, a prefix (or action) can be an input a , an output $\bar{a}$, or the silent action τ . A process can be the inactive process $\mathbf{0}$, a prefixed agent of the form $\alpha.Q$, or a nondeterministic choice among two processes $P + P'$. Agents can be

$$\begin{array}{c}
\alpha.Q \xrightarrow{\alpha} Q \text{ (ACT)} \quad \frac{Q_1 \xrightarrow{\alpha} Q'_1}{Q_1 + Q_2 \xrightarrow{\alpha} Q'_1} \text{ (SUM-L)} \quad \frac{Q_2 \xrightarrow{\alpha} Q'_2}{Q_1 + Q_2 \xrightarrow{\alpha} Q'_2} \text{ (SUM-R)} \\
\\
\frac{Q_1 \xrightarrow{\alpha} Q'_1}{Q_1 \parallel Q_2 \xrightarrow{\alpha} Q'_1 \parallel Q_2} \text{ (PAR-L)} \quad \frac{Q_2 \xrightarrow{\alpha} Q'_2}{Q_1 \parallel Q_2 \xrightarrow{\alpha} Q_1 \parallel Q'_2} \text{ (PAR-R)} \quad \frac{Q_1 \xrightarrow{\alpha} Q'_1 \quad Q_2 \xrightarrow{\bar{\alpha}} Q'_2}{Q_1 \parallel Q_2 \xrightarrow{\tau} Q'_1 \parallel Q'_2} \text{ (SYN)} \\
\\
\frac{Q \xrightarrow{\alpha} Q' \quad \alpha \notin \{a, \bar{a}\}}{(\nu a)Q \xrightarrow{\alpha} (\nu a)Q'} \text{ (R-RES)} \quad \frac{\mathcal{D}(X) = Q \quad Q \xrightarrow{\alpha} Q'}{X \xrightarrow{\alpha} Q'} \text{ (CONS)}
\end{array}$$

Fig. 1: CCS LTS

simple processes, a parallel composition of agents $Q \parallel Q'$, the restriction $(\nu a)Q$ that denotes that the name a is *locally bound* to the agent Q and cannot be used for communication outside of Q , or a process name X . Restriction is the only binder in CCS.

Let $\mathcal{P}$ denote the set of all CCS processes. For any process $P \in \mathcal{P}$, we write $\mathbf{n}(P)$ for the set of all names occurring in P , and $\mathbf{fn}(P)$ and $\mathbf{bn}(P)$ for the sets of free and bound names of P .

Definition 1 (CCS Semantics). *Given a set of defining equations $\mathcal{D} = \{X_i = Q_i\}$ where each X_i is a process constant and Q_i an agent, the operational semantics of CCS is defined as the LTS $(\mathcal{P}, \mathbf{Act}_\tau, \rightarrow)$, where the transition relation $\rightarrow$ is the smallest relation closed under the rules in Figure 1.*

We now comment on the rules in Figure 1. The ACT rule indicates that an agent of the form $\alpha.Q$ can perform α and evolves to Q . The PAR-L and PAR-R rules specify that either process in a parallel composition can execute an action independently while the other remains inert. The SYN rule regulates synchronisation, enabling two parallel processes to interact by simultaneously executing complementary actions (e.g., α and $\bar{\alpha}$). The RES rule enforces name restriction by preventing the bound name from being propagated outside the process Q . The CONS rule handles recursion by replacing a process identifier with its defining expression.

2.1 CCSK: CCS with communication keys

A general framework to obtain a reversible semantics for a calculus given in a particular SOS format is provided in [32]. The main ideas of such approach is to make every operator of a calculus *static*, since dynamic operator are forgetful, and to identify each action with a unique *key*. The result of applying such a framework on CCS is CCSK: that is CCS with communication keys. The syntax

of CCSK is then the following one.

$$\begin{aligned}
(\text{Actions}) \quad \alpha &::= a \mid \bar{a} \mid \tau & (\text{Prefixes}) \quad \pi &::= \alpha \mid \alpha[i] \\
(\text{Processes}) \quad P &::= \mathbf{0} \mid \pi.Q \mid P + P' & (\text{Agents}) \quad Q &::= P \mid Q \parallel Q' \mid (\nu a)Q \mid A
\end{aligned}$$

The only difference from CCS is the presence of the past-prefix operator $\pi[i].Q$, which denotes that the process Q has previously executed the action π , which has been bound to the fresh identifier i .

We write $\text{keys}(Q)$ for the set of keys of a process Q , given inductively by:

$$\begin{aligned}
\text{key}(Q) &= \emptyset \text{ if } \text{std}(Q) & \text{key}(X) &= \emptyset & \text{key}((\nu a)Q) &= \text{key}(Q) \\
\text{key}(\pi[i].Q) &= i \cup \text{key}(Q) & \text{key}(Q_1 + Q_2) &= \text{key}(Q_1 \parallel Q_2) &= \text{key}(Q_1) \cup \text{key}(Q_2)
\end{aligned}$$

A CCSK process is standard, noted $\text{std}(Q)$, when it does not contain communication keys, that is $\text{std}(Q) \iff \text{key}(Q) = \emptyset$.

Definition 2 (CCSK Semantics). *Given a set of defining equation $\mathcal{D}$, the operational semantics of CCS is defined as the LTS $(\mathcal{P}, \text{Act}_\tau, \rightarrow \cup \hookrightarrow)$, where the relations $\rightarrow$ and $\hookrightarrow$ are the smallest relations closed under the inference rules in the top and bottom sections, respectively, of Figure 2.*

We now comment on the inference rules and focus on the most complex forward-transition rules, noting that the backward rules are symmetric.

Rule ACT-1 converts a prefix into a past-action while preserving the prefix's information via a static operator. Rule ACT-2 allows a process under a past-action prefix to progress further. Rule SUM-L allows the execution of the left process of a choice. Unlike CCS, the non-selected branch (Q_2) is preserved rather than discarded. The requirement on Q_2 of being a standard CCS processes ensures that it has not execute any action. Rule PAR-L allows the left branch of a parallel composition to execute without synchronising with the right branch. The side condition ensures that the created key is not used in the right branch. Rule SYN, which accounts for synchronisation, requires the two complementary actions to share the same key.

Definition 3 (Strong Bisimulations). *A strong back and forth bisimulation between two LTSs $(Q_1, A_1, \rightarrow_{f,1} \cup \rightarrow_{b,1})$ and $(Q_2, A_2, \rightarrow_{f,2} \cup \rightarrow_{b,2})$ is a relation $\mathcal{R} \subseteq Q_1 \times Q_2$ where if $(q_1, q_2) \in \mathcal{R}$:*

1. $\forall q'_1 : q_1 \xrightarrow{\mu}_{f,1} q'_1 \text{ implies } \exists q'_2 : q_2 \xrightarrow{\mu}_{f,2} q'_2 \text{ and } (q'_1, q'_2) \in \mathcal{R}$
2. $\forall q'_1 : q_1 \xrightarrow{\mu}_{b,1} q'_1 \text{ implies } \exists q'_2 : q_2 \xrightarrow{\mu}_{b,2} q'_2 \text{ and } (q'_1, q'_2) \in \mathcal{R}$
3. $\forall q'_2 : q_2 \xrightarrow{\mu}_{f,2} q'_2 \text{ implies } \exists q'_1 : q_1 \xrightarrow{\mu}_{f,1} q'_1 \text{ and } (q'_1, q'_2) \in \mathcal{R}$
4. $\forall q'_2 : q_2 \xrightarrow{\mu}_{b,2} q'_2 \text{ implies } \exists q'_1 : q_1 \xrightarrow{\mu}_{b,1} q'_1 \text{ and } (q'_1, q'_2) \in \mathcal{R}$

If we restrict the relation $\mathcal{R}$ to the forward direction, by removing requirements (2) and (4) on the relation $\mathcal{R}$, and $\rightarrow_{b,1}, \rightarrow_{b,2}$ from the corresponding LTSs, we call $\mathcal{R}$ a strong bisimulation. We denote with $\sim_{bf}$ to be the largest strong back and forth bisimulation, and with $\sim$ the largest strong bisimulation.

$$\begin{array}{c}
\alpha.Q \xrightarrow{\alpha[i]} \alpha[i].Q \text{ (ACT1)} \quad \frac{Q \xrightarrow{\beta[j]} Q' \quad i \neq j}{\alpha[i].Q \xrightarrow{\beta[j]} \alpha[i].Q'} \text{ (ACT2)} \quad \frac{Q_1 \xrightarrow{\beta[j]} Q' \quad \text{std}(Q_2)}{Q_1 + Q_2 \xrightarrow{\beta[j]} Q'_1 + Q_2} \text{ (SUM-L)} \\
\\
\frac{Q_1 \xrightarrow{\beta[j]} Q'_1 \quad j \notin \text{key}(Q_2)}{Q_1 \parallel Q_2 \xrightarrow{\beta[j]} Q'_1 \parallel Q_2} \text{ (PAR-L)} \quad \frac{Q_1 \xrightarrow{\alpha[i]} Q'_1 \quad Q_2 \xrightarrow{\bar{\alpha}[i]} Q'_2}{Q_1 \parallel Q_2 \xrightarrow{\tau[i]} Q'_1 \parallel Q'_2} \text{ (SYN)} \\
\\
\frac{Q \xrightarrow{\alpha[i]} Q' \quad \alpha \notin \{a, \bar{a}\}}{(\nu a)Q \xrightarrow{\alpha[i]} (\nu a)Q'} \text{ (R-RES)} \quad \frac{\mathcal{D}(X) = Q \quad Q \xrightarrow{\alpha[i]} Q'}{X \xrightarrow{\alpha[i]} Q'} \text{ (CONS)} \\
\\
\alpha[i].Q \xrightarrow{\alpha[i]} \alpha.Q \text{ (ACT1}\bullet\text{)} \quad \frac{Q' \xrightarrow{\beta[j]} Q \quad i \neq j}{\alpha[i].Q' \xrightarrow{\alpha[i]} \alpha.Q} \text{ (ACT2}\bullet\text{)} \quad \frac{Q'_1 \xrightarrow{\beta[j]} Q_1 \quad \text{std}(Q_2)}{Q'_1 + Q_2 \xrightarrow{\beta[j]} Q_1 + Q_2} \text{ (SUM-L}\bullet\text{)} \\
\\
\frac{Q'_1 \xrightarrow{\beta[j]} Q_1 \quad j \notin \text{key}(Q_2)}{Q'_1 \parallel Q_2 \xrightarrow{\beta[j]} Q_1 \parallel Q_2} \text{ (PAR-L}\bullet\text{)} \quad \frac{Q'_1 \xrightarrow{\alpha[i]} Q_1 \quad Q'_2 \xrightarrow{\bar{\alpha}[i]} Q_2}{Q'_1 \parallel Q'_2 \xrightarrow{\tau[i]} Q_1 \parallel Q_2} \text{ (SYN}\bullet\text{)} \\
\\
\frac{Q \xrightarrow{\alpha[i]} Q' \quad \alpha \notin \{a, \bar{a}\}}{(\nu a)Q \xrightarrow{\alpha[i]} (\nu a)Q'} \text{ (R-RES}\bullet\text{)} \quad \frac{\mathcal{D}(X) = Q \quad Q' \xrightarrow{\alpha[i]} Q}{Q' \xrightarrow{\alpha[i]} X} \text{ (CONS}\bullet\text{)}
\end{array}$$

Fig. 2: CCSK LTS: symmetric rules are omitted

3 Nets

We start recalling definitions and notions on *multisets*. The symbol $\mathbb{N}$ indicates the set of natural numbers. A *multiset* over a set A is a function $f : A \rightarrow \mathbb{N}$. Multisets are assumed to be equipped with the standard operations of union (+) and difference (-). We write $f \subseteq f'$ if $f(a) \leq f'(a)$ for all $a \in A$. The multiset $\llbracket f \rrbracket$ is defined such that $\llbracket f \rrbracket(a) = 1$ if $f(a) > 0$ and $\llbracket f \rrbracket(a) = 0$ otherwise. We often confuse a multiset f with the set $\{a \in A \mid f(a) \neq 0\}$ when $f = \llbracket f \rrbracket$. In such cases, $a \in f$ denotes $f(a) \neq 0$, and $f \subseteq A$ signifies that $f(a) = 1$ implies $a \in A$ for all a . The underlying set of a multiset f , namely the one formed by the elements a with $f(a) > 0$, is precisely $\llbracket f \rrbracket$. Additionally, we will employ standard set operations like $\cap$, $\cup$, or $\setminus$. The set of all multisets over A is denoted as μA ; the symbol 0 stands for the unique multiset defined such that $\llbracket 0 \rrbracket = \emptyset$. We sometimes denote a multiset f as $\oplus_{a \in \llbracket f \rrbracket} f(a) \cdot a$, thus $a \oplus 2b$ would be the multiset f such that $f(a) = 1$ and $f(b) = 2$. Given a multiset f with $\#(f)$ we denote the number of its elements with their multiplicity, hence $\#(f) = \sum_{a \in \llbracket f \rrbracket} f(a)$.

A sequence of elements of a given set A is written as usual as $\rho = a_1 a_2 \cdots a_n$, with $a_i \in A$. The length of a sequence $\rho = a_1 a_2 \cdots a_n$, denoted by $\text{len}(\rho)$, is n . With ε we denote the empty sequence.

We recall the definition of net which we consider labelled, meaning that we associate each transition with a label from a fixed set.

Definition 4. A labelled net over $\mathbf{A}$ is the quintuple $N = \langle S, T, F, \lambda, \mathbf{A} \rangle$, where S and T are respectively the set of places and of transitions, and are such that $S \cap T = \emptyset$, $F \subseteq (S \times T) \cup (T \times S)$ is a set of arcs, and $\lambda : T \rightarrow \mathbf{A}$ is a labelling mapping.

Given a labelled net N and $x \in S \cup T$, we define the preset $\bullet x = \{y \mid (y, x) \in F\}$ and the postset $x^\bullet = \{y \mid (x, y) \in F\}$. If x is a place then $\bullet x$ and $x^\bullet$ are sets of transitions; analogously, if $x \in T$ then $\bullet x$ and $x^\bullet$ are sets of places.

Example 1. Consider the labelled net in Figure 3. The transition t_1 is labelled with a and has s_1 in its preset and s_1, s_2 in the postset. The transition t_4 is labelled with τ and $\bullet t_4 = \{s_2, s_4\}$ whereas $t_4^\bullet$ contains the place s_1 and s_4 .

We introduce the notion of *marking* for these nets and describe the token game.

Definition 5. A marked net is the pair $\mathbf{N} = (N, m)$ where $N = \langle S, T, F, \lambda, \mathbf{A} \rangle$ is a labelled net and $m : S \rightarrow \mathbb{N}$ is a marking.

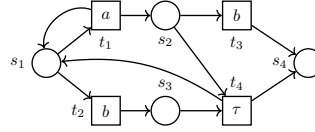


Fig. 3: A labelled net

We will often call *marked nets* simply nets. A transition t is *enabled* at a marking m , written $m[t]$, if $\forall s \in \bullet t. m(s) > 0$. If a transition t is enabled at a marking m then it may *fire* giving the marking m' defined as follows: $\forall s \in S. m(s) - 1$ if $s \in \bullet t \setminus t^\bullet$, $m(s) + 1$ if $s \in t^\bullet \setminus \bullet t$ and $m(s)$ for all others places. The firing of a transition t at m' producing the marking m' is denoted as $m[t] m'$. A *firing sequence* (shortened as *fs*) of (N, m) is a sequence σ of markings such that for each $1 \leq i < \text{len}(\sigma)$ there is a transitions t_i enabled at $\sigma(i)$ and $\sigma(i)[t_i] \sigma(i+1)$ and $\sigma(1)$ is m . A marking m' is *reachable* if there is a firing sequence ending with m' . With $\mathcal{M}_{(N, m)}$ we denote the set of reachable markings of the net (N, m) .

Definition 6. Let $\mathbf{N} = (N, m)$ be a net, where $N = \langle S, T, F, \lambda, \mathbf{A} \rangle$, then its marking graph is the tuple $M(N, m) = (\mathcal{M}_{(N, m)}, \mathbf{A}, \longrightarrow, m)$ where $\longrightarrow \subseteq \mathcal{M}_{(N, m)} \times \mathbf{A} \times \mathcal{M}_{(N, m)}$ is such that $(m', a, m'') \in \longrightarrow$, written $m' \xrightarrow{a} m''$, if there is a transition t such that $m'[t] m''$ and $\lambda(t) = a$.

4 CCS nets

We start by recalling the notion of CCS net as defined in [4] and inspired by previous works, for instance see [9].

Definition 7. A CCS net is a labeled net $N = \langle S, T, F, \lambda, A \rangle$ such that for all $t \in T$ $|\bullet t| \leq 2$ and if $|\bullet t| = 2$ then $\lambda(t) = \tau$.

A marked CCS net is the pair $H = (N, m)$ where N is a CCS net and $m : S \rightarrow \mathbb{N}$ is a marking.

The unique characteristic of this kind of nets is that transitions have either one incoming arc or two. Thus a place with various outgoing arcs represents the possible choice among various actions. A transition with two incoming arcs is a *synchronization*, and it involves just two places in their present, to match binary CCS synchronization.

We report the algorithm and main result presented in [4] adapted to our notation.

Algorithm 1: Encoding from CCS net to CCS process

```

Input   : CCS net  $\langle S, T, F, \lambda, A \rangle$  and marking  $m : S \rightarrow \mathbb{N}$ 
Output : CCS process  $Q$  and partial mapping of defining equations  $\mathcal{D}$ 
1  $\mathcal{D} \leftarrow$  Empty mapping of defining equations
2 for  $s \in S$  do  $\mathcal{D}(X_s) \leftarrow (Y_{t_1} + Y_{t_2} + \dots + Y_{t_k})$  where  $\{t_1, t_2, \dots, t_k\} = \{t \mid t \in s^\bullet\}$ 
3 for  $t \in \{t \mid t \in T \text{ and } |\bullet t| = 1\}$  do substitute  $Y_t$  in  $\mathcal{D}(X_{s^*})$  with
    $\lambda(t).(X_{s_1} \mid X_{s_2} \mid \dots \mid X_{s_k})$  where  $(s^*, t) \in F$  and  $\{s_1, s_2, \dots, s_k\} = \{s \mid s \in t^\bullet\}$ 
4  $A' \leftarrow \emptyset$ 
5 for  $t \in \{t \mid t \in T \text{ and } |\bullet t| = 2\}$  do
6    $A' \leftarrow A' \cup \{s_t\}$  where  $s_t$  is a fresh action
7   substitute  $X_t$  in  $\mathcal{D}(X_{s^*})$  with  $s_t.(X_{s_1} \mid X_{s_2} \mid \dots \mid X_{s_k})$ 
8   substitute  $X_t$  in  $\mathcal{D}(X_{s^{**}})$  with  $\overline{s_t}.0$  where
    $(s^*, t), (s^{**}, t) \in F$  and  $s^* \neq s^{**}$  and  $\{s_1, s_2, \dots, s_k\} = \{s \mid s \in t^\bullet\}$ 
9 end
10  $\{a_1, a_2, \dots, a_n\} \leftarrow A'$ 
11  $Q \leftarrow (\nu a_1)(\nu a_2) \dots (\nu a_n) \left( X_{s_1}^{m(s_1)} \mid X_{s_2}^{m(s_2)} \mid \dots \mid X_{s_{|S|}}^{m(s_{|S|})} \right)$ 
12 return  $(Q, \mathcal{D})$ 

```

Theorem 1 (Correctness of Algorithm 1). Let (N, m) be a CCS net system, and let the result of applying Algorithm 1 to N and m be the CCS process Q and defining equations $\mathcal{D}$. Then $M(N, m)$ is bisimilar to $LTS(Q, \mathcal{D})$.

5 Enriching nets

We want to generalize the notion of CCS net, well suited to encode CCS terms, to obtain a notion of CCSK net, that should be able to encode CCSK terms. We do need to add reversing transitions to a CCS nets and we have to enrich nets to have a way to keep track of the executed transitions. The first step is to establish what the tokens are in this kind of nets, and define the token game accordingly.

We fix an alphabet A of actions containing a special action τ and on that we form what are the *tokens* we will consider in the following. With $\langle\langle v_1, \dots, v_n \rangle\rangle$ we denote the tuple with n elements in some set, where the order does not count,

hence $\langle\langle v_1, \dots, v_i, \dots, v_j, \dots, v_n \rangle\rangle$ is the equivalent to $\langle\langle v_1, \dots, v_j, \dots, v_i, \dots, v_n \rangle\rangle$. $\langle\langle v_1, \dots, v_n \rangle\rangle \sim$ denotes the equivalence class with respect to the equivalence defined above, but we will constantly omit the subscript. The tuple with no element is $\langle\langle \rangle\rangle$.

Definition 8. Let A be a set of actions containing the special action τ , define a token as follows: $(\cdot, \langle\langle \rangle\rangle, \cdot)$ is a token, if $w_1, \dots, w_n$ are tokens, $a \in A$ is an action and $j \in \mathbb{N}$ is a number then $(a, \langle\langle w_1, \dots, w_n \rangle\rangle, j)$ is a token, and nothing else is a token. With Tok we denote the set of tokens.

Now tokens are not simply anonymous objects but each of them contains a code on how it has been produced. Given a token $w = (a, \langle\langle w_1, \dots, w_n \rangle\rangle, j)$, the mappings $\text{kt}(w) = j$, $\text{label}(w) = a$ and $[w] = (w_1, \dots, w_n)$ give respectively the key, the action labelling the transition producing that token and the tuple of tokens used for producing it. The *size* of a token is defined as follows: $\text{size}((\cdot, \langle\langle \rangle\rangle, \cdot)) = 0$ and $\text{size}((a, \langle\langle w_1, \dots, w_n \rangle\rangle, j)) = \max\{\text{size}(w_i) \mid 1 \leq i \leq n\} + 1$.

Example 2. Take $A = \{a, b, \tau\}$, tokens are $w_0 = (\cdot, \langle\langle \rangle\rangle, \cdot)$, $w_1 = (a, \langle\langle (\cdot, \langle\langle \rangle\rangle, \cdot) \rangle\rangle, 1)$, $w_2 = (b, \langle\langle (\cdot, \langle\langle \rangle\rangle, \cdot) \rangle\rangle, 1)$, $w_3 = (a, \langle\langle (\cdot, \langle\langle \rangle\rangle, \cdot), (b, \langle\langle (\cdot, \langle\langle \rangle\rangle, \cdot) \rangle\rangle, 1) \rangle\rangle, 2)$ and $w_4 = (\tau, \langle\langle (a, \langle\langle (\cdot, \langle\langle \rangle\rangle, \cdot) \rangle\rangle, 1), (b, \langle\langle (\cdot, \langle\langle \rangle\rangle, \cdot) \rangle\rangle, 2) \rangle\rangle, 3)$. Their size is $\text{size}(w_0) = 0$, $\text{size}(w_1) = 1 = \text{size}(w_2)$, $\text{size}(w_3) = 2$ as the size of the tokens $(\cdot, \langle\langle \rangle\rangle, \cdot)$ and $(b, \langle\langle (\cdot, \langle\langle \rangle\rangle, \cdot) \rangle\rangle, 1)$ is 0 and 1, and finally $\text{size}(w_4)$ is again 2.

The notion of net system changes accordingly: (N, m) with N labelled net and the marking m is now a mapping associating to each place a multiset of tokens, i.e. $m : S \rightarrow \mu\text{Tok}$.

To take into account the new notion of tokens, we have to reformulate the notions of enabling of a transition at a given marking, the firing of an enabled transition and accordingly the notion of firing sequence. Let N be a labelled net, and let $m : S \rightarrow \mu\text{Tok}$ be a marking. A transition $t \in T$ is *enabled* at m , written $m[t]$, if $\forall s \in \bullet t$ it holds that $m(s) \neq 0$, namely the place is marked. Let m be a marking and let t be a transition enabled at m . Then t may fire consuming the tokens w_{s_i} present in the places in $\bullet t = \{s_1, \dots, s_n\}$ and producing the token $w = (\lambda(t), \langle\langle w_{s_1}, \dots, w_{s_n} \rangle\rangle, i)$, where i is just a new index, in the places in the $t^\bullet$, giving the new marking m' . Formally, denoting with w_{s_i} the token used in the place $s_i \in \bullet t$ we obtain the marking m' , where for all $s \in S$

$$m'(s) = \begin{cases} m(s) - w_{s_i} + w & \text{if } s = s_i \in \bullet t \cap t^\bullet \\ m(s) - w_{s_i} & \text{if } s = s_i \in \bullet t \setminus t^\bullet \\ m(s) + w & \text{if } s = s_i \in t^\bullet \setminus \bullet t \\ m(s) & \text{otherwise} \end{cases}$$

The firing of a transition t at a marking m is denoted as usual: $m[t] m'$. The firing sequences are defined in the same way, as well as the notion of reachable marking and of marking graph. A marking $m : S \rightarrow \mu\text{Tok}$ is *initial* if $\forall s \in S$. $\llbracket m(s) \rrbracket \subseteq \{(\cdot, \langle\langle \rangle\rangle, \cdot)\}$ and a place s is unmarked if $m(s) = 0$.

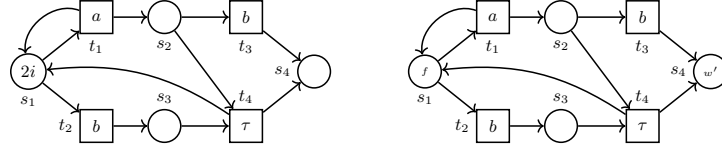


Fig. 4: A labeled net with the initial marking and after the firing of t_1, t_2 and t_4 . The tokens are $i = (\cdot, \langle\langle\cdot\rangle\rangle, \cdot)$, $w = (a, \langle\langle i \rangle\rangle, 1)$, $w' = (\tau, \langle\langle (a, \langle\langle i \rangle\rangle, 1), (b, \langle\langle i \rangle\rangle, 2) \rangle\rangle, 3)$ and $f \in \mu\text{Tok}$ is the multiset $w \oplus w'$.

Example 3. Consider the labelled nets in Figure 4. On the right only the place s_1 is marked and the marking is $m(s_1) = (\cdot, \langle\langle\cdot\rangle\rangle, \cdot) \oplus (\cdot, \langle\langle\cdot\rangle\rangle, \cdot)$ and all the other places are such that $m(s_j) = 0$ for $j \in \{2, 3, 4\}$. At this marking the enabled transitions are t_1 and t_2 . Assume the t_1 is executed. The resulting marking m_1 is $m_1(s_1) = (\cdot, \langle\langle\cdot\rangle\rangle, \cdot) \oplus (a, \langle\langle (\cdot, \langle\langle\cdot\rangle\rangle, \cdot) \rangle\rangle, 1)$, $m_1(s_2) = (a, \langle\langle (\cdot, \langle\langle\cdot\rangle\rangle, \cdot) \rangle\rangle, 1)$ and the other places are unmarked. Thus $m[t_1]m_1$. At m_1 the transitions t_1, t_2 and t_3 are enabled. By executing t_2 using the token $(\cdot, \langle\langle\cdot\rangle\rangle, \cdot)$ we obtain the marking m_2 where $m_2(s_1) = (a, \langle\langle (\cdot, \langle\langle\cdot\rangle\rangle, \cdot) \rangle\rangle, 1)$, $m_2(s_2) = m_1(s_2)$, $m_2(s_3) = (b, \langle\langle (\cdot, \langle\langle\cdot\rangle\rangle, \cdot) \rangle\rangle, 2)$. At this marking all the transitions are enabled and we decide to fire t_4 . We get the marking m_3 shown in Figure 4 in the net on the right. The firing sequence is $m[t_1]m_1[t_2]m_2[t_4]m_3$. If instead of executing t_4 at m_2 we would execute t_3 the reached marking would be $m'(s_1) = (a, \langle\langle (\cdot, \langle\langle\cdot\rangle\rangle, \cdot) \rangle\rangle, 1)$, $m'(s_2) = 0$, $m'(s_3) = (b, \langle\langle (\cdot, \langle\langle\cdot\rangle\rangle, \cdot) \rangle\rangle, 2)$ and $m'(s_4) = (b, \langle\langle (a, \langle\langle (\cdot, \langle\langle\cdot\rangle\rangle, \cdot) \rangle\rangle, 1) \rangle\rangle, 3)$.

The second step is to enrich a labelled net with *key* places whose purpose is to keep a precise record of the fired transitions.

Definition 9. A *key labelled net* $K = (N, S_k)$ is a pair where $N = \langle S, T, F, \lambda, A \rangle$ is a labelled net such that there exists a subset $S_k \subseteq S$ of places, called *key places*, such that for all $t \in T$ $|t^\bullet \cap S_k| = 1$ and for all $s \in S_k$. $s^\bullet = \emptyset$ and $|\bullet s| = 1$.

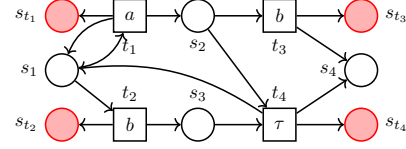


Fig. 5: A key labelled net.

A direct consequence of this definition is that $|T| = |S_k|$. The key place associated to a transition, which is unique, conveys the idea of the key associated to a transition when it is executed.

Example 4. In Figure 5 it is shown a key labelled net. Removing the key places we obtain the labelled net depicted in Figure 4 on the left.

Each labelled net can be transformed into a key labelled net.

Proposition 1. Let $N = \langle S, T, F, \lambda, A \rangle$ be a labelled net, and let $S_k = \{s_t \mid t \in T\}$ be a new set of places. Then $K(N) = (N', S_k)$ where $N' = \langle S \cup S_k, T, F \cup \{(t, s_t) \mid t \in T\}, \lambda, A \rangle$, is a key labelled net.

Key places can also be removed from a key labelled net.

Proposition 2. *Let $K = (N, S_k)$ be a key labelled net with $N = \langle S, T, F, \lambda, A \rangle$. Then $\mathcal{G}(K) = \langle S \setminus S_k, T, F \setminus \{(t, s) \mid s \in S_k, t \in T, \lambda, A\} \rangle$ is a labelled net.*

Observe that given a labelled net N $\mathcal{G}(\mathcal{K}(N)) = N$ up to renaming of key places and given K be a key labelled net we have $\mathcal{K}(\mathcal{G}(K)) = K$ again up to renaming of key places. It is straightforward to observe that key places do not influence the token game.

Proposition 3. *Let $K = (N, S_k)$ be a key labelled net, m a marking and $t \in T$ a transition. Then $m[t]$ iff $m|_{S \setminus S_k}[t]$ and if $m[t] m'$ then $m|_{S \setminus S_k}[t] m'|_{S \setminus S_k}$ where $m|_{S \setminus S_k} : S \setminus S_k \rightarrow \mu\text{Tok}$ is the restriction of m to the places in $S \setminus S_k$.*

If $m|_{S \setminus S_k}[t] m'|_{S \setminus S_k}$ then we have that there is a marking $\tilde{m} : S \rightarrow \mu\text{Tok}$ such that $m|_{S \setminus S_k} = \tilde{m}$ and $\tilde{m}[t] \tilde{m}'$ with $m'|_{S \setminus S_k} = \tilde{m}'$.

As each key place is associated to a given transition, we can, using the name of the transition, find the associated key place.

Definition 10. *Let $K = (N, S_k)$ be a key labelled net and let $t \in T$ be a transition. Then with s^t we denote the key place $s^t \in S_k$ such that $\bullet s^t = \{t\}$. The key place associated to a transition t is denoted with $\text{keypl}(t)$.*

Definition 11. *Let $K = (N, S_k)$ be a key labelled net where $N = \langle S, T, F, \lambda, A \rangle$ and let $m : S \rightarrow \mu\text{Tok}$ be a marking. We say that m is initial iff (a) $\forall s \in S. \llbracket m(s) \rrbracket \subseteq \{(\cdot, \langle \rangle), (\cdot, \cdot)\}$ and (b) $\forall s \in S_k. m(s) = 0$.*

A marking is initial iff all the keyplaces are unmarked and if another place is marked, then it contains the *initial* token $(\cdot, \langle \rangle, \cdot)$.

We introduce now *reversing* transitions. The idea is simple: given a transition t we add a transition u that has the effect of *reversing* the execution of t .

Definition 12. *A reversible labelled net with reversing transitions $U \subseteq T$ and key places $S_k \subseteq S$ is the triple $R = (\langle S, T, F, \lambda, A \rangle, S_k, U)$ where*

1. $\langle S, T, F, \lambda, A \rangle$ is a labelled net,
2. $(\langle S, T \setminus U, F \cap (\{(s, u) \mid s \in S, u \in U\} \cup \{(u, s) \mid s \in S, u \in U\}), \lambda, A \rangle, S_k)$ is a key labelled net, and
3. $U \subseteq T$ and $\forall u \in U. \exists! t \in T \setminus U$ such that (a) $\bullet u = t^\bullet$ and $u^\bullet = \bullet t$, and (b) $\lambda(t) = \lambda(u)$.

In a reversible labelled net we have a designated subset U of reversing transitions that are such that, for each reversing transition, there is exactly one *forward* transition (not reversing one) such that their presets and postsets are exchanged.

Example 5. In Figure 6 we depict a reversible labeled net and the associated labelled net without the reversing transitions.

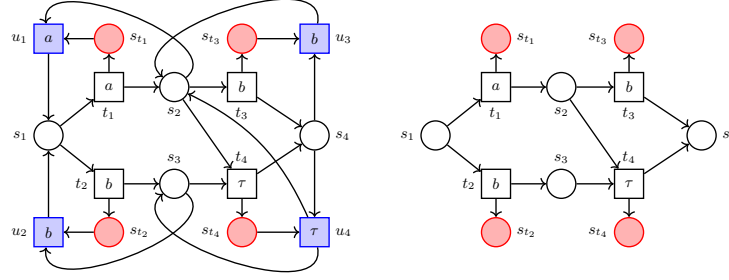


Fig. 6: A reversible labeled net (on the left) and the associated key labelled net.

Enabling and firing a reversing transition: Executing a reversing transition is different from executing a *normal* one. First of all, it has to undo the effects of firing a transition t , and the token produced in the postset of the transition t depends on the tokens consumed, the labelling of t and on a fresh index, thus a reversing transition u has first to check that there is the *same* token in each of the places in its preset.

Definition 13. Let $R = (\langle S, T, F, \lambda, A \rangle, S_k, U)$ be a reversible labelled net and let $u \in U$ be a reversing transition, and let $t \in T \setminus U$ the corresponding forward transition. Then u is enabled at the marking m whenever there exist a token $w \in m(s)$ such that $\text{label}(w) = \lambda(t)$ and $w \in m(s)$ for each $s \in \bullet u$ and for each $w' \in [w]$ there exists a place $s \in u^\bullet$ such that either there $\exists t \in \bullet s$ and $w' \in \text{keypl}(t)$ or $w' = (\cdot, \langle \cdot, \cdot \rangle, \cdot)$. The enabling of a reversing transition u at a marking m is denoted with $m \Vdash u$.

Example 6. Consider the reversible labelled net depicted in Figure 7 on the left and the marking $m(s_1) = m(s_2) = m(s_3) = m(s_{t_3}) = 0$, $m(s_4) = w = m(s_{t_4})$, $m(s_{t_1}) = (a, \langle (\cdot, \langle \cdot, \cdot \rangle, \cdot) \rangle, 1)$ and $m(s_{t_2}) = (b, \langle (\cdot, \langle \cdot, \cdot \rangle, \cdot) \rangle, 3)$, where $w = (\tau, \langle (a, \langle (\cdot, \langle \cdot, \cdot \rangle, \cdot) \rangle, 1), (b, \langle (\cdot, \langle \cdot, \cdot \rangle, \cdot) \rangle, 2) \rangle, 3)$. The reverse transition u_4 is enabled, thus $m \Vdash u_4$, as in all the places in its preset (namely s_4 and s_{t_4}) the token $w_3 = (\tau, \langle (a, \langle (\cdot, \langle \cdot, \cdot \rangle, \cdot) \rangle, 1), (b, \langle (\cdot, \langle \cdot, \cdot \rangle, \cdot) \rangle, 2) \rangle, 3)$ is present and the *forward* transitions putting a token in the places in postset of $u_4 = \{s_2, s_3\}$, namely t_1 and t_2 , are such that $\text{keypl}(t_1) = s_{t_1}$ contains the token $(a, \langle (\cdot, \langle \cdot, \cdot \rangle, \cdot) \rangle, 1)$ and $m(\text{keypl}(t_2)) = m(s_{t_2}) = (b, \langle (\cdot, \langle \cdot, \cdot \rangle, \cdot) \rangle, 2)$.

The execution of the reversing transition u at the marking m is formalized in the following definition.

Definition 14. Let $R = (\langle S, T, F, \lambda, A \rangle, S_k, U)$ be a reversible labelled net and let $u \in U$ be a reversing transition. Let m be a marking such that $m \Vdash u$ under the token w . Then u is may fire at the marking m giving the marking m' , written

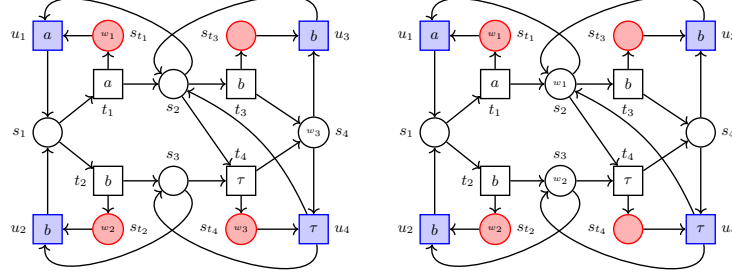


Fig. 7: A reversible labelled net (on the left) after the firing of t_1, t_2 and t_4 starting from an initial marking $m(s_1) = (\cdot, \langle\langle\cdot\rangle\rangle, \cdot) \oplus (\cdot, \langle\langle\cdot\rangle\rangle, \cdot)$. On the right, the marking after undoing the transition t_4 .

as $m \parallel u \rangle m'$ where

$$m'(s) = \begin{cases} m(s) - w + w' & \text{if } s \in \bullet u \cap u^\bullet \text{ with } w' \in \text{keypl}(t) \text{ for some } t \in \bullet s \\ m(s) + w' & \text{if } s \in u^\bullet \setminus u^\bullet \text{ with } w' \in \text{keypl}(t) \text{ for some } t \in \bullet s \\ m(s) - w & \text{if } s \in \bullet u \setminus u^\bullet \\ m(s) & \text{otherwise} \end{cases}$$

Example 7. In the reversible labelled net depicted on the right in Figure 7 it is shown the marking after the reverse firing of u_4 .

Given a reversible labelled net $R = (\langle S, T, F, \lambda, A \rangle, S_k, U)$, with $\mathcal{B}(R)$ we denote the key labelled net $(\langle S, T \setminus U, F \cap (\{(s, u) \mid s \in S, u \in U\} \cup \{(u, s) \mid s \in S, u \in U\}), \lambda, A \rangle, S_k)$ as prescribed in Definition 12.

We end this part defining what is a marking graph of a reversible labelled net. Consider a reversible labelled net $R = (\langle S, T, F, \lambda, A \rangle, S_k, U)$, and a marking m . Then the execution of a transition (either forward or reversing) is defined as $m \parallel x \rangle, m'$ if either $m[x] m'$ if $x \notin U$ or $m \parallel x \rangle m'$ if $x \in U$. The firing sequence is defined accordingly and the reachable markings as well. The set of the reachable markings of a reversible labelled net $R = (\langle S, T, F, \lambda, A \rangle, S_k, U)$ with initial marking m is denoted as usual with $\mathcal{M}_{(R, m)}$.

Definition 15. Let $R = (\langle S, T, F, \lambda, A \rangle, S_k, U)$ be a reversible labelled net, then its marking graph is the tuple $M(R, m) = (\mathcal{M}_{(R, m)}, A, \longrightarrow, m)$ where $\longrightarrow \subseteq \mathcal{M}_{(R, m)} \times A \times \mathcal{M}_{(R, m)}$ is such that $(m', a, m'') \in \longrightarrow$, written $m' \xrightarrow{a} m''$, if there is a transition $x \in T$ such that $m' \parallel x \rangle, m''$ and $\lambda(x) = a$

6 CCSK nets

We have now all the ingredients to be able to define what a CCSK net is.

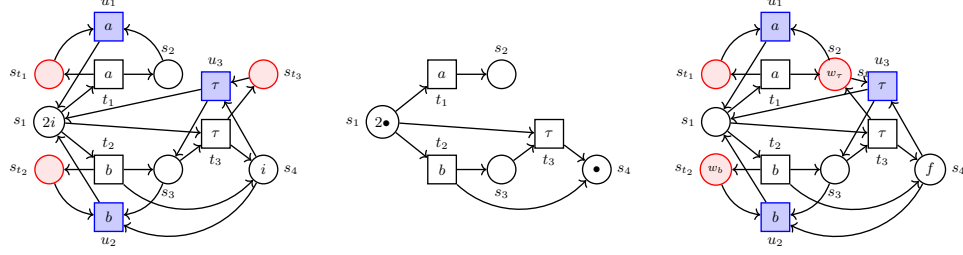


Fig. 8: An initial *ccsk-net* $G = (N, S_k, U, m_{in}, m)$, the associated CCS net system $(\mathcal{G}(\mathcal{B}((N, S_k, U))), \tilde{m})$ where $\tilde{m}(s) = \#(m(s))$ for all $s \in S \setminus S_k$ and the *ccsk-net* after the execution of the transitions t_2 and t_3 from the initial marking where s_1 carries the multiset $2i$ and the place s_4 the multiset i , where i is the initial token $(\cdot, \langle\langle\cdot\rangle\rangle, -)$, w_b is the token $(b, \langle\langle i \rangle\rangle, 1)$, w_τ is $(\tau, \langle\langle (b, \langle\langle i \rangle\rangle, 1), i \rangle\rangle, 2)$ and f is the multiset of tokens $i \oplus w_b \oplus w_\tau$.

Definition 16. A CCSK net $P = (\langle S, T, F, \lambda, A \rangle, S_k, U)$ is a reversible labelled net such that $\mathcal{G}(\mathcal{B}(P))$ is CCS net.

In fact, a CCSK net is just a reversible labelled net where the labelled net with only the forward transitions and without the key places is precisely a CCS net.

Definition 17. A CCSK net system (*ccsk-net for short*) is the tuple $G = (N, S_k, U, m_{in}, m)$ where (N, S_k, U) is a CCSK net, $m_{in} : S \rightarrow \mu\text{Tok}$ is an initial marking of $\mathcal{B}((N, S_k, U))$ and $m : S \rightarrow \mu\text{Tok}$ is a marking.

The requirement that m_{in} should be an initial marking of the key labelled net $\mathcal{B}((N, S_k, U))$ is motivated by the fact that we have to keep track of the starting CCSK process.

Definition 18. A *ccsk-net* $G = (N, S_k, U, m_{in}, m)$ is initial when $m = m_{in}$.

Example 8. In Figure 8 we depict, on the left, a *ccsk-net* and in the middle the associated CCS net system. The initial tokens are $i = (\cdot, \langle\langle\cdot\rangle\rangle, \cdot)$. The key places are those drawn in red. On the right the same *ccsk-net* after the executions of t_2 and t_3 (and the only marked key places are indeed s_{t_2} and s_{t_3}).

Proposition 4. Let $G = (N, S_k, U, m_{in}, m)$ be a *ccsk-net*, then $(\mathcal{G}(\mathcal{B}((N, S_k, U))), \tilde{m})$, where $\tilde{m} : S \setminus S_k \rightarrow \mathbb{N}$ is a marking defined as $\tilde{m}(s) = \#(m(s))$, is a CCS net.

7 From CCSK net to CCSK processes

We present now the algorithm that, given a *ccsk-net* $G = (\langle S, T, F, \lambda, A \rangle, S_k, U, m_{in}, m)$, gives as output a CCSK process and a partial mapping of defining equations (abbreviated as *pmde*).

An *enhanced* prefix π is a prefix of the form $\pi ::= \alpha \mid \alpha[i] \mid \alpha[s]$ where s belongs to a set S_k of key places, that are sometimes called *key holders*. An *enhanced CCSK* term is a CCSK term where the prefixes are enhanced. The mapping ψ transform an enhanced CCSK term into a CCSK one by removing all the key holders. Thus $\psi(a[i].b[s].\mathbf{0} + a[s'].\mathbf{0}) = a[i].b.\mathbf{0} + a.\mathbf{0}$.

Lists of elements are written by listing its elements separated with a “,” in square brackets and $[]$ denotes the empty list. Given a list $[e_1; e_2; \dots; e_n]$ and an element e , with $e :: [e_1; e_2; \dots; e_n]$ we denote the list $[e; e_1; e_2; \dots; e_n]$. **hd** and **tl** give respectively the head and the tail of a list, and **@** is the usual concatenation operation on lists. The mapping **mklist** receives an element e and a number i and produce a list with i copies of e . Thus $\mathbf{mklist}(e, 3) = [e; e; e]$.

The algorithm receives a **ccsk-net** and works as follows: first it runs an adapted version of the Algorithm 1 using $(\mathcal{B}((N, S_k, U)), m_{in})$ as inputs which produces a list of enriched CCSK terms and a **pmde** where each equation is on enriched CCSK terms, together with a list of variables and a set of actions (that are the fresh one to be restricted), and then using the marking m on the key places it transforms the partial mapping of defining equations into another partial mapping of defining equations and a CCSK terms without enhanced prefixes, assigning to CCSK terms the correct index. Auxiliary functions used in the algorithms are briefly described here.

- **getkeys** takes an enhanced CCSK term of the form $\alpha_1[s_1].(X_1^1 \mid \dots \mid X_1^{n_1}) + \dots + \alpha_k[s_k].(X_k^1 \mid \dots \mid X_k^{n_k})$ and returns the set $\{s_1, \dots, s_k\}$ of key holders appearing in the enhanced CCSK term,
- **minkeys** takes a set of key holders $\{s_1, \dots, s_k\}$, a mapping $m : S_k \rightarrow \mu\text{Tok}$ and either a token w' or $\bullet$ and gives back a pair (s_i, w) such that $w \in m(s_i)$, $[w]$ contains w' if w' is not $\bullet$, and $\text{size}(w)$ is the minimal among those in $m(s_j)$ for each $1 \leq j \leq k$ satisfying the previous conditions,
- **getvar** takes an enhanced CCSK term $\alpha_1[s_1].(X_1^1 \mid \dots \mid X_1^{n_1}) + \dots + \alpha_k[s_k].(X_k^1 \mid \dots \mid X_k^{n_k})$ and a key holder s_i and returns the variable in $(X_i^1 \mid \dots \mid X_i^{n_i})$,
- **tolist** takes a set of elements and transforms it in a list,
- **toterm** takes a non empty list of variables and gives the parallel compositions of all the variables in the list,
- **Var** takes an (enhanced) CCSK term Q and returns the set of variables appearing in it,
- **findmin** takes a set $\mathcal{S}$ of key holders, a mapping m and a token list l_{tok} and gives a set of pairs (s, w) where s is a key holder and w a token containing the head of the list of tokens, and
- **clean** takes a **pmde** $\mathcal{D}$ where some terms may be enhanced and gives the **pmde** $\mathcal{D}$ where the terms are just CCSK ones.

In the following we write the algorithms involved. The empty mapping of defining equations is denoted with $\mathcal{D}_\emptyset$. Algorithm 2 is the main algorithm and after initializing the list of variables, the set of restricted actions, the empty list of tokens, it run the adaptation of the Algorithm 1, which is the Algorithm 3. The Algorithm 3 returns also a list of variables, and in a **while** loop the Algorithm 2 first applies the Algorithm 4 to get an enhanced CCSK term Q , with

Algorithm 2: From CCSK net system to CCSK process and defining equations

```

Input      : CCSK net system  $G = (\langle S, T, F, \lambda, A \rangle, S_k, U, m_{in}, m)$ 
Output    : CCSK process  $Q$  and a pmde  $\mathcal{D}$ 
1  $l_{var} \leftarrow [], A \leftarrow \emptyset, l_{var}^{res} \leftarrow [], \mathcal{D} \leftarrow \mathcal{D}_\emptyset, l_{tok} \leftarrow [], (\mathcal{D}, l_{var}, A) \leftarrow \text{Algorithm 3}(G)$ 
2 while  $l_{var} \neq []$  do
3    $(\mathcal{D}, Q) \leftarrow \text{Algorithm 4}(\mathcal{D}, \text{hd}(l_{var}))$ 
4    $\mathcal{D}(X) \leftarrow Q$  where  $X$  is a new variable
5    $l_{var}^{res} \leftarrow X :: l_{var}^{res}$ 
6    $(\mathcal{D}, m, l_{tok}) \leftarrow \text{Algorithm 5}(\mathcal{D}, X, m, l_{tok})$ 
7    $l_{var} \leftarrow \text{tl}(l_{var})$ 
8 end
9  $\{s_{t_1}, \dots, s_{t_n}\} \leftarrow A, \mathcal{D} \leftarrow \text{clean}(\mathcal{D})$ 
10  $Q \leftarrow (\nu s_{t_1})(\nu s_{t_2}) \dots (\nu s_{t_n}) \text{toterm}(l_{var}^{res})$ 
11 return  $(Q, \mathcal{D})$ 

```

variable renamed, associated to the first element and, because of the renaming of variables, a pmde $\mathcal{D}$. Then it associate Q to a fresh variable which is inserted in a list and it runs the main routine, namely Algorithm 5. The variables in the list of results are transformed in a CCSK term which is given as result together with the defining equations $\mathcal{D}$.

Algorithm 3: Encoding from CCSK net systems G to $(\mathcal{D}, l, A')$

```

Input      : CCSK net system  $G = (\langle S, T, F, \lambda, A \rangle, S_k, U, m_{in}, m)$ 
Output    : pmde  $\mathcal{D}$ , a list of variables  $l$  and a set of fresh actions  $A'$ 
1  $\mathcal{D} \leftarrow \mathcal{D}_\emptyset, A' \leftarrow \emptyset$ 
2 for  $s \in S \setminus S_k$  do  $\mathcal{D}(X_s) \leftarrow (Y_{t_1} + Y_{t_2} + \dots + Y_{t_k})$  where
    $\{t_1, t_2, \dots, t_k\} = \{t \mid t \in T \setminus U \wedge t \in s^\bullet\}$ 
3 for  $t \in \{t \mid t \in T \setminus U \text{ and } |\bullet t| = 1\}$  do substitute  $Y_t$  in  $\mathcal{D}(X_{s^*})$  with
    $\lambda(t)[\text{keypl}(t)].(X_{s_1} \mid X_{s_2} \mid \dots \mid X_{s_n})$  where  $(s^*, t) \in F$  and  $s_i \in t^\bullet \setminus S_k$ 
4 for  $t \in \{t \mid t \in T \setminus U \text{ and } |\bullet t| = 2\}$  do
5    $A' \leftarrow A' \cup \{a_t\}$  where  $a_t$  is a fresh action
6   substitute  $Y_t$  in  $\mathcal{D}(X_{s^*})$  with  $a_t[\text{keypl}(t)].(X_{s_1} \mid X_{s_2} \mid \dots \mid X_{s_n})$ 
7   substitute  $Y_t$  in  $\mathcal{D}(X_{s^{**}})$  with  $\bar{a}_t[\text{keypl}(t)].0$  where
      $(s^*, t), (s^{**}, t) \in F$  and  $s^* \neq s^{**}$  and  $s_i \in t^\bullet \setminus S_k$ 
8 end
9  $l = \text{mklist}(X_{s_1}, \widetilde{m_{in}}(s_1)) @ \text{mklist}(X_{s_2}, \widetilde{m_{in}}(s_2)) @ \dots @ \text{mklist}(X_{s_{|S|}}, \widetilde{m_{in}}(s_{|S|}))$ 
10 return  $(\mathcal{D}, l, A')$ 

```

The Algorithm 4 simply adds to the defining equations new variable associated to equations appearing in an enhanced CCSK term, and the new variables are associated to the corresponding definitions. As the new variable are added, nothing is lost.

The main routine is Algorithm 5. This part uses the mapping from key places (key holders) to assign the correct key to the action in an enhanced CCSK term Q identified by its defining equation. It receives also a list of tokens. It runs Algorithm 4 to make copies of the variables in the enhanced CCSK term. If the list of token is empty then it checks the key holders in Q . If Q does not have any then it finishes, otherwise it takes one among the shortest keys. The reason

Algorithm 4: Renaming variables and updating $\mathcal{D}$

Input : pmde $\mathcal{D}$ and an enhanced CCSK term Q
Output : pmde $\mathcal{D}$ and an enhanced CCSK term Q
1 **for** $X_i \in \text{Var}(Q)$ **do** $\mathcal{D}(X'_i) \leftarrow Q\{X'_i/X_i\}$ **where** X'_i is a fresh variable
2 **return** $(\mathcal{D}, Q)$

Algorithm 5: Assignment of keys to enhanced CCSK terms

Input : A pmde $\mathcal{D}$, a variable X , a mapping $m : S_k \rightarrow \mu\text{Tok}$ and a list l_{tok} of tokens
Output : A pmde $\mathcal{D}$, a mapping $m : S_k \rightarrow \mu\text{Tok}$ and a list of tokens l'_{tok}

```

1 if  $l_{\text{tok}} = []$  then
2    $S \leftarrow \text{getkeys}(Q)$ 
3   if  $S \neq \emptyset$  then
4      $(s, w) \leftarrow \text{findmin}(m, S, \bullet)$ 
5      $\mathcal{V} \leftarrow \text{getvar}(Q, s)$ 
6      $\mathcal{D}(X) \leftarrow \psi(Q\{\{s/\text{kt}(w)\}\})$ 
7     if  $[w]$  is  $(w_1, w_2)$  then  $m(s) = m(s) - w + (\text{label}(w), w_2, \text{kt}(w))$  else
8        $m(s) = m(s) - w$ 
9      $l_{\text{tok}} \leftarrow w :: l_{\text{tok}}$ 
10    for  $X \in \mathcal{V}$  do
11       $(\mathcal{D}, Q) \leftarrow \text{Algorithm 4}(\mathcal{D}, \mathcal{D}(X)), \mathcal{D}(X) \leftarrow Q$ 
12       $(\mathcal{D}, m, l_{\text{tok}}) \leftarrow \text{Algorithm 5}(\mathcal{D}, X, m, l_{\text{tok}})$ 
13    end
14  else
15     $S \leftarrow \text{getkeys}(Q)$ 
16    if  $S \neq \emptyset$  then
17       $(s, w) \leftarrow \text{findmin}(m, S, \text{hd}(l'_{\text{tok}}))$ 
18       $\mathcal{V} \leftarrow \text{getvar}(Q, s)$ 
19       $\mathcal{D}(X) \leftarrow \psi(Q\{\{s/\text{kt}(w)\}\})$ 
20      if  $[w]$  is  $(w_1, w_2)$  then  $m(s) = m(s) - w + (\text{label}(w), w_2, \text{kt}(w))$  else
21         $m(s) = m(s) - w$ 
22       $l_{\text{tok}} \leftarrow w :: l_{\text{tok}}$ 
23      for  $X \in \mathcal{V}$  do
24         $(\mathcal{D}, Q) \leftarrow \text{Algorithm 4}(\mathcal{D}, \mathcal{D}(X)), \mathcal{D}(X) \leftarrow Q$ 
25         $(\mathcal{D}, m, l_{\text{tok}}) \leftarrow \text{Algorithm 5}(\mathcal{D}, X, m, l_{\text{tok}})$ 
26      end
27     $l_{\text{tok}} \leftarrow \text{tl}(l_{\text{tok}})$ 
28  end
29 return  $(\mathcal{D}, m, l_{\text{tok}})$ 

```

for picking one among the shortest keys depends on the need of allocating first the initial keys. The marking is updated and the exploration of the CCSK term continues using the variables *after* the prefix considered. The other parts of the term are made, so to speak, inactive using the mapping ψ which is applied to the term $Q\{\{s/\text{kt}(w)\}\}$ which is the enhanced CCSK term where the key holder s is replaced with the correct index.

The exploration uses the same Algorithm 5 to the designated variables. If the list of tokens is not empty then the token in the head of the list is used to find another token, if present, which has that to-

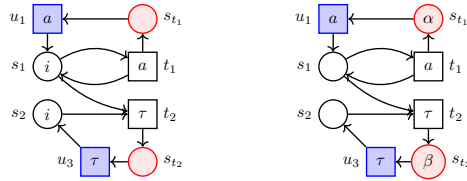


Fig. 9: An initial ccsk-net $G = (N, S_k, U, m_{in}, m)$ and the same ccsk-net after the execution of transitions a and τ .

ken in it, meaning that it is something that is *after* in the CCSK term. Then it does as above, updating the mapping m . Observe that the tokens under inspection are removed and in the case they are tokens representing a synchronization then the token is replaced with one of a smaller size. It is worth to observe that Algorithm 5 converges.

Example 9. In Figure 9 depicts an initial CCSK net system (on the left) and the same CCSK net system after the execution of a and τ . The tokens are $\alpha = (a, \langle i \rangle, 1)$ and $\beta = (\tau, \langle \alpha, i \rangle, 2)$. When we execute the Algorithm 3 we get $\mathcal{D}(X_{s_1}) = a[s_{t_1}].X_{s_1} + c[s_{t_2}].0$ and $\mathcal{D}(X_{s_2}) = \bar{c}[s_{t_2}].0$, the list $[X_{s_1}; X_{s_2}]$ and a set of new actions $A = \{c\}$. The list is not empty and we start with X_{s_1} , and, after applying Algorithm 4, we have that equations $\mathcal{D}(X''_{s_1}) = a[s_{t_1}].X'_{s_1} + c[s_{t_2}].0$ and $\mathcal{D}(X'_{s_1}) = a[s_{t_1}].X_{s_1} + c[s_{t_2}].0$ are added to $\mathcal{D}$. The new variable X''_{s_1} is added to the list of results and the Algorithm 5 is invoked, with an empty list of tokens and the marking $m(s_{t_1}) = \alpha$ and $m(s_{t_2}) = \beta$. The shortest token among the ones associated to the key holders of $a[s_{t_1}].X'_{s_1} + c[s_{t_2}].0$, namely $\{s_{t_1}, s_{t_2}\}$, is α , and $\lfloor \alpha \rfloor$ is a singleton, therefore $\mathcal{D}(X''_{s_1}) = a[1].X'_{s_1} + c[s_{t_2}].0$, $\mathcal{V} = \{X'_{s_1}\}$, the marking is updated, now being $m(s_{t_1}) = 0$, and the token α is added to the list of tokens. The Algorithm 4 is used to update $a[1].X'_{s_1} + c[s_{t_2}].0$, associated to X''_{s_1} , obtaining $a[s_{t_1}].X'''_{s_1} + c[s_{t_2}].0$ and $\mathcal{D}(X'''_{s_1}) = a[1].X'_{s_1} + c[s_{t_2}].0$. The new invocation of Algorithm 5 is with a list of tokens containing the token α , the variable X'''_{s_1} and the updated marking. The set of key holders contains just s_{t_2} , furthermore the token β is such that $\lfloor \beta \rfloor$ contains α . Furthermore the set of variables is now empty, and the result is that $\mathcal{D}(X'''_{s_1})$ is updated to $a[1].X'_{s_1} + c[2].0$, β is added to the list of tokens and the marking associated to s_{t_2} is updated removing β and adding $(\tau, \langle i \rangle, 2)$. At this point β is removed from the list of tokens. Now we end also the first invocation of this algorithm, and we go back to analyze the list $[X_{s_2}]$. We create a new variable X'_{s_2} which is added to the resulting list that now is $[X'_{s_2}; X'_{s_1}]$ and $\mathcal{D}(X'_{s_2}) = \bar{c}[s_{t_2}].0$. The Algorithm 5 will put in the term $\bar{c}[s_{t_2}].0$ the proper key obtaining $\mathcal{D}(X'_{s_2}) = \bar{c}[2].0$ and the token $(\tau, \langle i \rangle, 2)$ will be removed from $m(s_{t_2})$. The final result will be $Q = (\nu c)(X'_{s_2} \mid X'_{s_1})$ and the $\mathcal{D}$ constructed so far.

We now state the correspondence between CCSK nets and CCSK.

Theorem 2 (Correctness of Algorithm 2). *Let $G = (\langle S, T, F, \lambda, A \rangle, S_k, U, m_{in}, m)$ be a ccsk-net, and let the result of applying Algorithm 2 to G be the CCSK process Q and defining equations $\mathcal{D}$. Then $M(G, m)$ is bisimilar to $LTS(Q, \mathcal{D})$.*

8 Conclusions

Inspired by the work in [4] we have provided a class of Petri nets, called CCSK nets, which correspond to CCSK processes. We also have provided an algorithm which given a CCSK net construct the corresponding CCSK term.

There are several directions for future works. First we want to improve our results, and to implement the provided algorithm in LEAN⁴. This on one side would allow us to benchmark our results, on the other hand would allow us to mechanise the correctness proof.

Also, a generalisation of CCS nets to multi-synchronisation nets should be possible. A direct consequence of this would be to obtain a CCSK with multi-synchronisation. Another direction, would be to connect CCSK nets with event structures, by finding a suitable notion of (reversible) event structure corresponding to CCSK.

References

1. T. Abdellatif, J. Kornas, and J.-B. Stefani. Reengineering J2EE servers for automated management in distributed environments. *IEEE Distributed Syst. Online*, 8(11), 2007.
2. B. Aman, G. Ciobanu, R. Glück, R. Kaarsgaard, J. Kari, M. Kutrib, I. Lanese, C. A. Mezzina, L. Mikulski, R. Nagarajan, I. C. C. Phillips, G. M. Pinna, L. Prigioniero, I. Ulidowski, and G. Vidal. Foundations of reversible computation. In I. Ulidowski, I. Lanese, U. P. Schultz, and C. Ferreira, editors, *Reversible Computation: Extending Horizons of Computing - Selected Results of the COST Action IC1405*, volume 12070 of *Lecture Notes in Computer Science*, pages 1–40. Springer, 2020.
3. C. H. Bennett. Notes on the history of reversible computation. *IBM J. Res. Dev.*, 44(1):270–278, 2000.
4. B. Bogø, A. Burattin, and A. Scalas. Encoding petri nets into CCS. In I. Castellani and F. Tiezzi, editors, *Coordination Models and Languages - 26th IFIP WG 6.1 International Conference, COORDINATION 2024*, volume 14676 of *Lecture Notes in Computer Science*, pages 38–55. Springer, 2024.
5. E. Bruneton, T. Coupaye, M. Leclercq, V. Quéma, and J.-B. Stefani. The FRAC-TAL component model and its support in java. *Softw. Pract. Exp.*, 36(11-12):1257–1284, 2006.
6. V. Danos and J. Krivine. Reversible communicating systems. In P. Gardner and N. Yoshida, editors, *CONCUR 2004 - Concurrency Theory*, volume 3170 of *Lecture Notes in Computer Science*, pages 292–307. Springer, 2004.
7. V. Danos and J. Krivine. Transactions in RCCS. In M. Abadi and L. de Alfaro, editors, *CONCUR 2005*, volume 3653 of *Lecture Notes in Computer Science*, pages 398–412. Springer, 2005.
8. G. Fabbretti, I. Lanese, and J.-B. Stefani. Reversibility with holes - (work in progress). In T. Æ. Mogensen and L. Mikulski, editors, *Reversible Computation - 16th International Conference, RC 2024*, volume 14680 of *Lecture Notes in Computer Science*, pages 69–74. Springer, 2024.
9. R. Gorrieri and C. Versari. A process calculus for expressing finite place/transition petri nets. In S. B. Fröschle and F. D. Valencia, editors, *Proceedings 17th International Workshop on Expressiveness in Concurrency, EXPRESS’10*, volume 41 of *EPTCS*, pages 76–90, 2010.
10. V. Koutavas, M. Gazda, and M. Hennessy. Distinguishing between communicating transactions. *Inf. Comput.*, 259(1):1–30, 2018.

⁴ <https://lean-lang.org/>

11. J. Krivine. A verification technique for reversible process algebra. In R. Glück and T. Yokoyama, editors, *Reversible Computation, 4th International Workshop, RC 2012. Revised Papers*, volume 7581 of *Lecture Notes in Computer Science*, pages 204–217. Springer, 2012.
12. S. Kuhn and I. Ulidowski. Modelling of DNA mismatch repair with a reversible process calculus. *Theor. Comput. Sci.*, 925:68–86, 2022.
13. P. Lami, I. Lanese, J.-B. Stefani, C. S. Coen, and G. Fabbretti. Reversible debugging of concurrent erlang programs: Supporting imperative primitives. *J. Log. Algebraic Methods Program.*, 138:100944, 2024.
14. I. Lanese, M. Lienhardt, C. A. Mezzina, A. Schmitt, and J.-B. Stefani. Concurrent Flexible Reversibility. In M. Felleisen and P. Gardner, editors, *Programming Languages and Systems*, Lecture Notes in Computer Science, pages 370–390, Berlin, Heidelberg, 2013. Springer.
15. I. Lanese, D. Medic, and C. A. Mezzina. Static versus dynamic reversibility in CCS. *Acta Informatica*, 58(1-2):1–34, 2021.
16. I. Lanese, C. A. Mezzina, A. Schmitt, and J.-B. Stefani. Controlling Reversibility in Higher-Order Pi. In J.-P. Katoen and B. König, editors, *CONCUR 2011 – Concurrency Theory*, pages 297–311, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.
17. I. Lanese, C. A. Mezzina, and J.-B. Stefani. Reversing Higher-Order Pi. In P. Gastin and F. Laroussinie, editors, *CONCUR 2010 - Concurrency Theory*, Lecture Notes in Computer Science, pages 478–493, Berlin, Heidelberg, 2010. Springer.
18. I. Lanese, C. A. Mezzina, and J.-B. Stefani. Reversibility in the higher-order π -calculus. *Theoretical Computer Science*, 625:25–84, 2016.
19. S. Lenglet, A. Schmitt, and J. Stefani. Characterizing contextual equivalence in calculi with passivation. *Inf. Comput.*, 209(11):1390–1433, 2011.
20. S. Lenglet, A. Schmitt, and J.-B. Stefani. Howe’s method for calculi with passivation. In M. Bravetti and G. Zavattaro, editors, *CONCUR 2009 - Concurrency Theory, 20th International Conference*, volume 5710 of *Lecture Notes in Computer Science*, pages 448–462. Springer, 2009.
21. S. Lenglet, A. Schmitt, and J.-B. Stefani. Normal bisimulations in calculi with passivation. In L. de Alfaro, editor, *Foundations of Software Science and Computational Structures, 12th International Conference, FOSSACS 2009*, volume 5504 of *Lecture Notes in Computer Science*, pages 257–271. Springer, 2009.
22. H. C. Melgratti, C. A. Mezzina, and G. M. Pinna. A petri net view of covalent bonds. *Theor. Comput. Sci.*, 908:89–119, 2022.
23. H. C. Melgratti, C. A. Mezzina, and G. M. Pinna. A reversible perspective on petri nets and event structures. *ACM Trans. Comput. Log.*, 25(4):1–38, 2024.
24. H. C. Melgratti, C. A. Mezzina, and G. M. Pinna. A truly concurrent semantics for reversible CCS. *Log. Methods Comput. Sci.*, 20(4), 2024.
25. H. C. Melgratti, C. A. Mezzina, and I. Ulidowski. Reversing place transition nets. *Log. Methods Comput. Sci.*, 16(4), 2020.
26. C. A. Mezzina, R. Schlatte, R. Glück, T. Haulund, J. Hoey, M. H. Cservenka, I. Lanese, T. Æ. Mogensen, H. Siljak, U. P. Schultz, and I. Ulidowski. Software and reversible systems: A survey of recent activities. In I. Ulidowski, I. Lanese, U. P. Schultz, and C. Ferreira, editors, *Reversible Computation: Extending Horizons of Computing - Selected Results of the COST Action IC1405*, volume 12070 of *Lecture Notes in Computer Science*, pages 41–59. Springer, 2020.
27. C. A. Mezzina, F. Tiezzi, and N. Yoshida. Checkpoint-based rollback recovery in session programming. *Log. Methods Comput. Sci.*, 21(1):2, 2025.

28. R. Milner. *A Calculus of Communicating Systems*, volume 92 of *Lecture Notes in Computer Science*. Springer, 1980.
29. A. Philippou and K. Psara. A collective interpretation semantics for reversing petri nets. *Theor. Comput. Sci.*, 924:148–170, 2022.
30. A. Philippou and K. Psara. Reversible computation in nets with bonds. *J. Log. Algebraic Methods Program.*, 124:100718, 2022.
31. I. Phillips, I. Ulidowski, and S. Yuen. A reversible process calculus and the modelling of the ERK signalling pathway. In R. Glück and T. Yokoyama, editors, *Reversible Computation, 4th International Workshop, RC 2012. Revised Papers*, volume 7581 of *Lecture Notes in Computer Science*, pages 218–232. Springer, 2012.
32. I. C. C. Phillips and I. Ulidowski. Reversing algebraic process calculi. *J. Log. Algebraic Methods Program.*, 73(1-2):70–96, 2007.
33. W. Reisig. *Petri Nets: An Introduction*, volume 4 of *EATCS Monographs on Theoretical Computer Science*. Springer, 1985.
34. A. Schmitt and J.-B. Stefani. The m-calculus: a higher-order distributed process calculus. In A. Aiken and G. Morrisett, editors, *Conference Record of POPL 2003*, pages 50–61. ACM, 2003.
35. A. Schmitt and J.-B. Stefani. The kell calculus: A family of higher-order distributed process calculi. In C. Priami and P. Quaglia, editors, *Global Computing, IST/FET International Workshop, GC 2004, Rovereto, Italy, March 9-12, 2004, Revised Selected Papers*, volume 3267 of *Lecture Notes in Computer Science*, pages 146–178. Springer, 2004.
36. L. Seinturier, P. Merle, R. Rouvoy, D. Romero, V. Schiavoni, and J.-B. Stefani. A component-based middleware platform for reconfigurable service-oriented architectures. *Softw. Pract. Exp.*, 42(5):559–583, 2012.
37. J.-B. Stefani. A calculus of kells. In R. Focardi and G. Zavattaro, editors, *2nd EATCS Workshop on Foundations of Global Computing, FGC 2003, Satellite Event of ICALP 2003*, volume 85 of *Electronic Notes in Theoretical Computer Science*, pages 40–60. Elsevier, 2003.
38. J.-B. Stefani and M. Vassor. Encapsulation and sharing in dynamic software architectures: The hypercell framework. In J. A. Pérez and N. Yoshida, editors, *Formal Techniques for Distributed Objects, Components, and Systems - 39th IFIP WG 6.1 International Conference, FORTE 2019*, volume 11535 of *Lecture Notes in Computer Science*, pages 242–260. Springer, 2019.
39. M. Vassor and J.-B. Stefani. Checkpoint/rollback vs causally-consistent reversibility. In J. Kari and I. Ulidowski, editors, *Reversible Computation - 10th International Conference, RC 2018, Leicester, UK, September 12-14, 2018, Proceedings*, volume 11106 of *Lecture Notes in Computer Science*, pages 286–303. Springer, 2018.